

Introducción a Eclipse

Table of Contents

Objetivos	1
Prerrequisitos	2
1. Introducción	2
2. Instalación de Eclipse	2
2.1. Descarga e instalación de Eclipse IDE	2
3. Configuración del WORKSPACE	4
4. Instalación de Python	7
5. Instalación de plugins de Eclipse	8
5.1. PyDev en Eclipse	8
5.2. Configurar Python en Eclipse	8
5.3. JGrasp plugin para Eclipse	9
5.4. GitHub Copilot plugin para Eclipse	9
6. Entorno de desarrollo Eclipse	10
6.1. Zona de ventanas	10
6.2. Perspectivas	10
6.3. Proyectos	11
6.3.1. Creación de un nuevo proyecto "en blanco"	11
6.3.2. Importar un proyecto existente	12
6.3.3. Clonar un proyecto desde un repositorio	12
6.4. Creación de un programa Java	12
6.5. CTRL + Espacio = Autocompletar	14
6.6. Compilar y detectar errores	14
6.7. Icono de Bombilla = Autocorregir	15
6.8. Ejecución de una aplicación Java	15
6.9. Ejecución con argumentos	17
7. Conclusiones	19

Objetivos

- Instalación de Eclipse for Java Developers
- Configuración del WORKSPACE de Eclipse
- Familiarizarse con el entorno de desarrollo Eclipse

Prerrequisitos

- Ninguno

1. Introducción

Eclipse es un entorno de desarrollo integrado (IDE), realizado con el lenguaje de programación Java, que fue originalmente diseñado para la realización de programas con este lenguaje de programación. Con el tiempo, su uso y popularidad se han ido extendiendo entre los programadores profesionales. En la actualidad ofrece varias distribuciones y extensiones (*plugins*) que permiten desarrollar programas en varios lenguajes de programación. En nuestro caso vamos a usar la distribución **Eclipse for Java Developers**, que es la que está orientada al desarrollo de programas en Java.

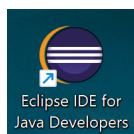


Figura 1. Eclipse for Java Developers

2. Instalación de Eclipse

2.1. Descarga e instalación de Eclipse IDE

1. Descarga la distribución **Eclipse IDE for Java Developers** desde aquí: <https://www.eclipse.org/downloads/packages/>. Busca la distribución **Eclipse IDE for Java Developers**, y haz clic en el enlace de descarga para tu sistema operativo (Windows, Linux o Mac).

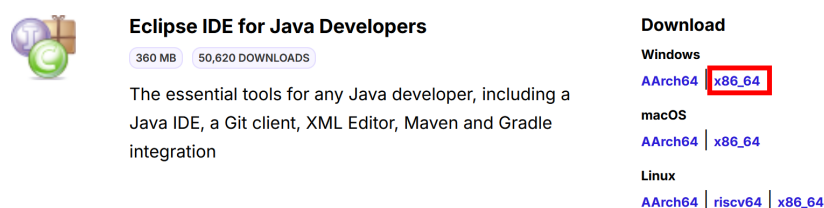


Figura 2. Descarga de Eclipse for Java Developers

2. En tu carpeta **Descargas** se habrá descargado un archivo comprimido **.zip**, por ejemplo **eclipse-java-2026-09-R-win32-x86_64.zip**.

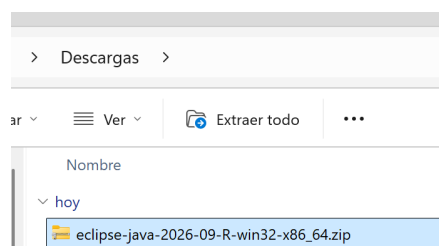


Figura 3. Archivo .zip con Eclipse

3. Crea una carpeta llamada **C:\ECLIPSES**. A continuación, extrae (descomprime) el contenido del archivo **.zip** en la carpeta que acabas de crear **C:\ECLIPSES**: para ello, sobre el archivo zip, botón derecho, extraer todo...
4. Ejecuta Eclipse haciendo doble clic sobre el archivo **eclipse.exe** que se encuentra en la carpeta donde se ha descomprimido el archivo **.zip**.

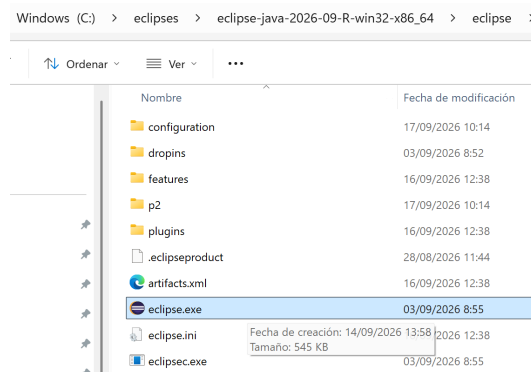


Figura 4. Ejecuta Eclipse

Si no sabes cual es el archivo **eclipse.exe** activa la visualización de las extensiones de los archivos en tu explorador de archivos del sistema operativo Windows. Para ello, en el explorador selecciona **_Ver > Mostrar > Extensiones de nombres de archivo**

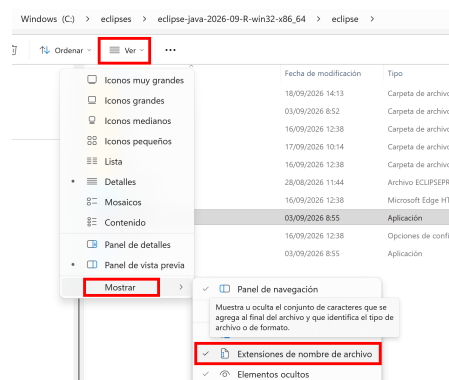


Figura 5. Mostrar extensiones de archivos

Enhorabuena ! Eclipse se está iniciando.



Figura 6. Eclipse se está iniciando



Como método alternativo, también puedes instalar Eclipse desde el gestor de paquetes de tu sistema operativo. En Windows, por ejemplo, puedes instalarlo con Chocolatey (<https://chocolatey.org/>), ejecutando en el terminal (**cmd**) el comando **choco install eclipse-java-oxygen -y**

3. Configuración del WORKSPACE

Cuando ejecutamos Eclipse, lo primero que aparece es una pequeña pantalla de inicio (lo que se conoce en inglés como *splash screen*, figura [Eclipse se está iniciando](#)) que desaparece al cabo de un ratito y, de inmediato, aparece una ventana preguntándonos por el directorio o carpeta donde queremos ubicar nuestro espacio de trabajo (**workspace**), carpeta en la que se van a guardar nuestros proyectos. La recomendación es la siguiente:

- Con el explorador de archivos del sistema operativo, crea la carpeta **WORKSPACES** en el disco y carpeta de tu ordenador donde desees trabajar (por ejemplo, entro de **Mis Documentos**, o bien en el raíz del disco **C:...** en fin, donde tú te organices mejor. Dentro de ella, crea la carpeta del tipo **InicialesAsignaturaAño**, por ejemplo: **IP2026**, **MP2026**, **PC2026**, etc. Esa será la carpeta de trabajo de Eclipse, por ejemplo: **C:\WORKSPACES\IP2026**.
- En Eclipse, configura el workspace para que apunte a esa carpeta. Para ello eligelá si aun está esperando a elegir al workspace, o bien accede al menú: **File > Switch workspace > Other ...**

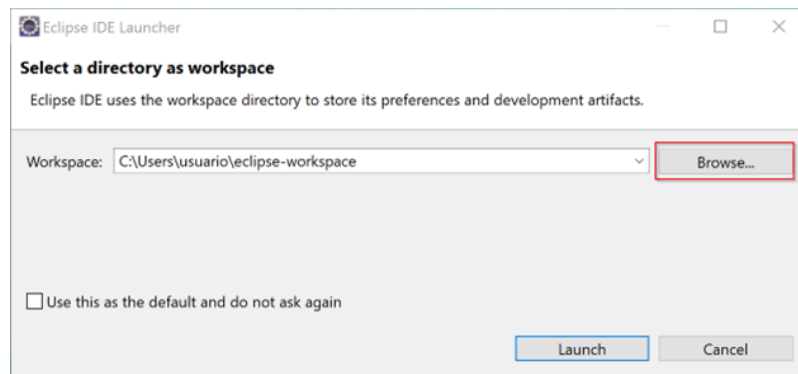


Figura 7. Switch workspace

- Con **Browse**, selecciona la carpeta del workspace indicada previamente. Si estás trabajando en tu propio portátil, marca la casilla “*Use this as the default and do not ask again*”. Pero si estás trabajando en un **PC del Aula de Informática**, te recomiendo que **no marques** esa casilla.
- La primera vez que se abre Eclipse, debe aparecer también una ventana: *Windows Defender Exclusion Check*. Aquí, se debe marcar la opción: *Exclude Eclipse SDK from being scanned to improve performance*.

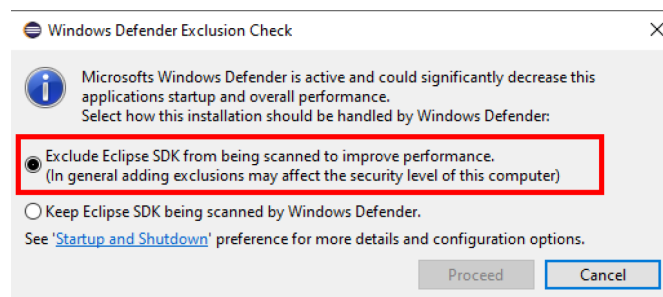


Figura 8. Exclude Eclipse SDK from being scanned

- Tras ello, pulsa **Launch**. Eclipse se reiniciará. Haz clic en **Hide** para comenzar a trabajar.

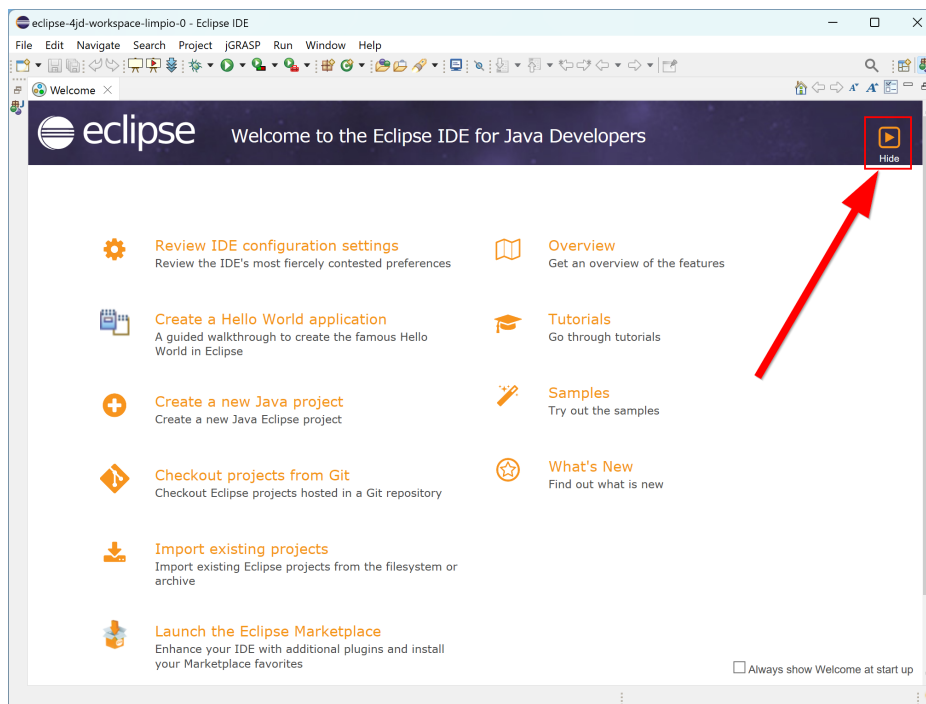


Figura 9. Hide Welcome view

- A continuación, debes asegurarte de que la codificación está configurada a **UTF-8** como predeterminada para tus proyectos Java. Para ello, en el menú *Window > Preferences* (si estás usando un Mac: *Eclipse > Preferences*). Despliega en el menú de la izquierda *General > Workspace*. Comprueba que está marcado *Text file encoding, Other: UTF-8*. Guarda con *Apply and Close*

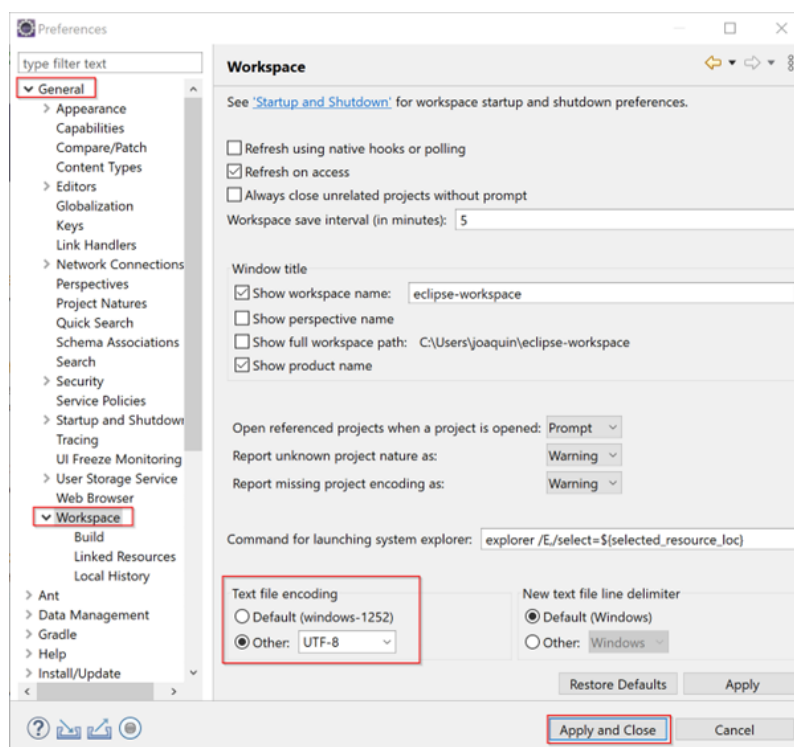


Figura 10. Configuración de la codificación UTF-8

¡Enhorabuena! Tu Eclipse ya se ha iniciado correctamente por primera vez.

Si trabajas en un **PC del Aula** no olvides que, antes de finalizar la clase, debes **guardar tu workspace** en un pendrive o en una carpeta en la nube, para no perder el trabajo realizado. Los PCs de las Aulas de Informática están configurados para que cuando se apagan se borran todos los archivos creados por el usuario, por lo que si no guardas tu workspace antes de apagar el ordenador, tu trabajo se perderá !!!



Además, la siguiente semana cuando llegues al Aula tendrás que traer tu workspace con el trabajo realizado de la semana anterior y configurar de nuevo Eclipse para que lo use.

En cualquier caso, como vamos a trabajar con repositorios GitHub, tu trabajo no lo perderás porque se quedará guardado en el repositorio de GitHub. Para ello, no olvides siempre hacer **Commit and Push** antes de apagar el PC. Lo que tendrás que hacer cada día que uses el PC del Aula será volver a **configurar el workspace** en el PC del Aula y **clonar tu repositorio**.

4. Instalación de Python

1. En el sitio web oficial de Python diríjase a la sección de descargas (Downloads) y descargue el instalador ejecutable más reciente para Windows (<https://www.python.org/downloads/windows/>), se recomienda la versión 3.14.x (ej. `python-3.14.7-amd64.exe`).



Figura 11. Descarga de Python para Windows

2. Ejecute el archivo descargado.



En la primera pantalla del instalador, asegúrese de marcar la casilla "Add python.exe to PATH" en la parte inferior. Esto permitirá que el sistema y Eclipse reconozcan Python de forma global.

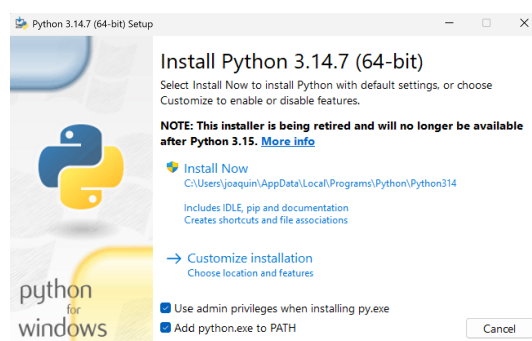


Figura 12. Marcar la casilla "Add python.exe to PATH"

3. Haga clic en "Install Now" y complete el asistente de instalación.
4. Para verificar que se instaló correctamente, abra la consola de comandos de Windows (cmd) y ejecute:

```
python --version
```

5. Instalación de plugins de Eclipse

5.1. PyDev en Eclipse

Abra su entorno de desarrollo Eclipse.

En la barra de menú superior, diríjase a Help (Ayuda) > Eclipse Marketplace...

En el cuadro de búsqueda (Find), escriba PyDev y presione la tecla Enter o haga clic en el botón Go.

Localice el complemento denominado PyDev - Python IDE for Eclipse.

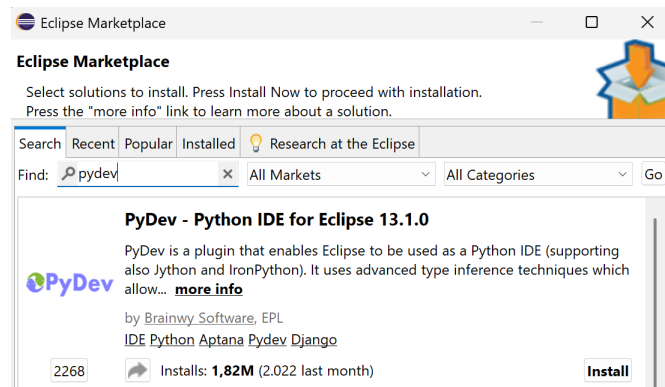


Figura 13. Instalar PyDev

Haga clic en el botón *Install* asociado a este complemento.

Siga los pasos del asistente de instalación, acepte los términos de la licencia y presione Finish.

Si el entorno solicita un reinicio para aplicar los cambios, confirme y permita que Eclipse se reinicie.

5.2. Configurar Python en Eclipse

Una vez instalados ambos componentes, es necesario indicarle a Eclipse qué versión de Python utilizar.

1. Dentro de Eclipse, vaya al menú Window > Preferences (Preferencias).
2. En el árbol de opciones de la izquierda, despliegue PyDev > Interpreters y seleccione Python Interpreter.
3. Eclipse buscará automáticamente la instalación de Python realizada en el primer apartado. Si no la detecta automáticamente, haga clic en New... > Choose from list , y elija la versión de Python instalada en su sistema operativo.

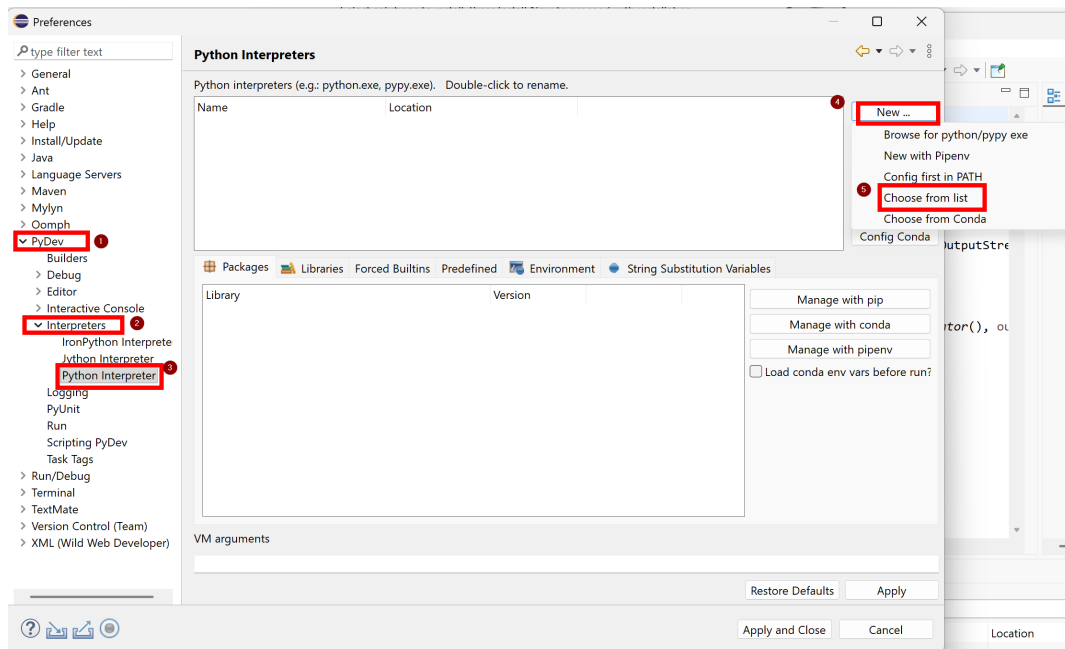


Figura 14. Seleccionar Python

5. Aplique los cambios haciendo clic en Apply and Close

5.3. JGrasp plugin para Eclipse

1. Abrir Eclipse for Java Developers.
2. Go to "Help" > "Install New Software..." > "Add..." button > Location: https://www.jgrasp.org/eclipse_plugin > Hit "Next", "Next".
3. Siga los pasos del asistente de instalación, acepte los términos de la licencia y presione Finish.

5.4. GitHub Copilot plugin para Eclipse

Abrir Eclipse for Java Developers. Help > Eclipse Marketplace > Find: Copilot > Github Copilot > Install > I accept the terms ... > Finish > Marcar Authority / Update site > Trust Selected > Restart Now

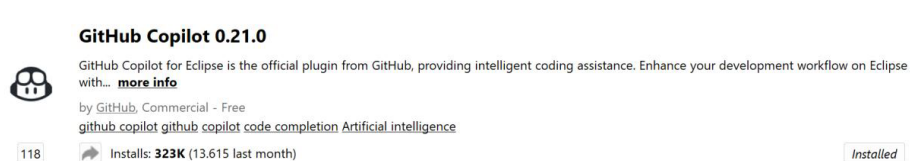


Figura 15. Instalar el plugin de Copilot

6. Entorno de desarrollo Eclipse

6.1. Zona de ventanas

Eclipse se abre con las siguientes ventanas:

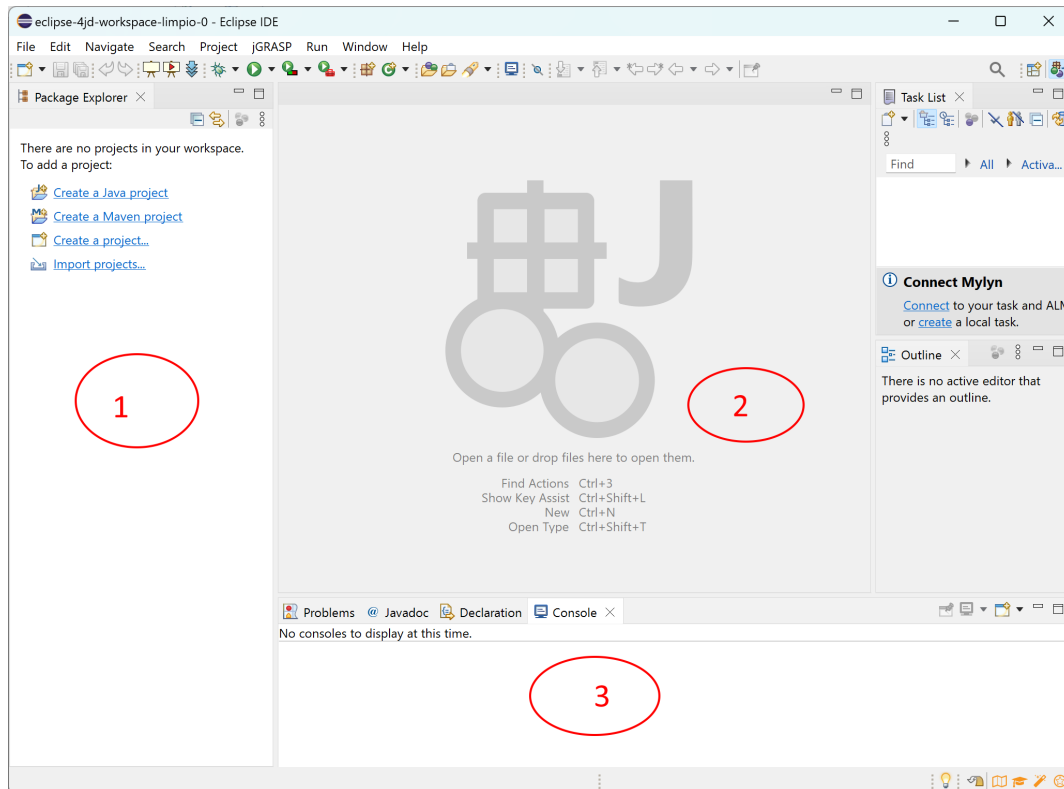


Figura 16. Áreas de Eclipse

1. **Área 1:** Explorador de Paquetes. Siguiendo una estructura arborescente (forma de árbol), en esta área se muestran todos los proyectos, paquetes y clases que se almacenan en el workspace seleccionado.
2. **Área 2:** Editor de código. Zona de trabajo donde escribiremos el código fuente.
3. **Área 3:** Esta área está compuesta por múltiples "pestañas", entre las que destacamos la pestaña *Problems*, que mostrará aquellos mensajes de error en tiempo de compilación. Y, por otra parte, está la pestaña *Console*, que será donde se muestren los mensajes de error o bien los mensajes a los que da lugar nuestro programa. Otras pestañas se utilizarán más adelante.



Si no se visualiza la pestaña *Console* activa la misma desde el menú *Window > Show View > Console*.

6.2. Perspectivas

Las áreas visibles anteriores corresponden a la perspectiva Java. Una **perspectiva** de Eclipse es una agrupación de vistas y editores de manera que den apoyo a una actividad completa del proceso de desarrollo software. Los editores normalmente permiten realizar una tarea completa, las vistas proporcionan funciones de apoyo o auxiliares tales como mostrar información, requerir datos, etc. Las perspectivas pueden seleccionarse haciendo clic en los iconos de perspectiva en la esquina

superior derecha o eligiendo *Window > Open Perspective* del menú.

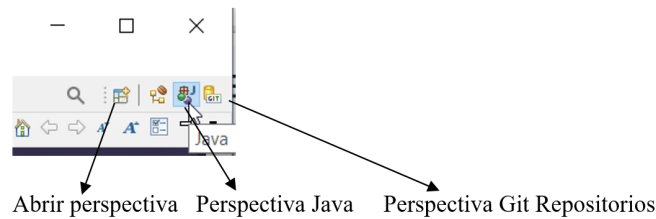


Figura 17. Cambiar de perspectiva

Perspectivas que usaremos:

- **Java:** esta perspectiva se centra en tareas de programación, mostrando paquetes, clases, métodos y atributos en sus vistas asociadas.
- **Debug:** relacionada con la tarea de depuración. Se centra en los procesos ejecutados, puntos de ruptura, variables, salida, etc.
- **Git:** esta perspectiva se centra en el trabajo con repositorios. Permitirá establecer conexiones con diferentes repositorios, acceder a los mismos, navegar por ellos, etc.

6.3. Proyectos

En el entorno de desarrollo Eclipse todo archivo se almacena dentro de un **proyecto** (*project*). Esto quiere decir que todo documento, carpeta, archivo de código fuente (**.java**) y código compilado (**.class**) tiene que estar contenido dentro de un proyecto. Así pues, el primer paso antes de usar Eclipse para programar en Java es comprender la estructura de proyectos de Eclipse.

6.3.1. Creación de un nuevo proyecto "en blanco"

Puedes hacer la prueba de crear un nuevo proyecto Java: nos situaremos en el área del explorador de paquetes y seleccionamos botón derecho, *New > Java Project* y nos guiaremos por el asistente. El nombre del proyecto Java puede ser, por ejemplo, *PrimerProyecto*. Desmarca la opción *Create module-info.java file*, y pulsa *Finish*.

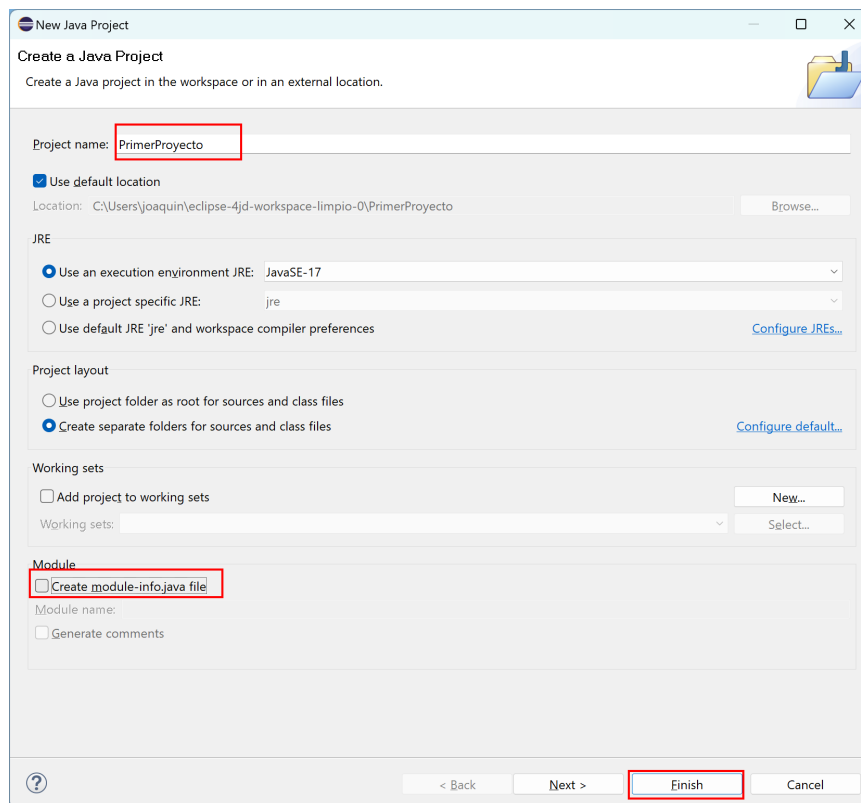


Figura 18. Creación de un proyecto Java

6.3.2. Importar un proyecto existente

También podríamos **importar un proyecto ya existente**. Para ello seleccionaremos igualmente el botón derecho del ratón, *Import > General > Existing Projects into Workspace > Select archive file* e indicamos donde se encuentra.

6.3.3. Clonar un proyecto desde un repositorio

A partir del curso 2023/2024 no tendrás que crear tu proyecto personal. La forma de trabajar con tu proyecto personal de la asignatura será **clonar tu proyecto desde tu repositorio personal en GitHub**. Dicho proyecto ya contiene los archivos necesarios para comenzar a programar en las prácticas de la asignatura. Para clonar tu proyecto, sigue los pasos descritos en el documento **Uso básico de Git y GitHub en Eclipse**.

6.4. Creación de un programa Java

Una vez que dispongas de un proyecto en el que empezar a trabajar, ya puedes crear tu primer programa Java.

1. En el proyecto Java recién creado, creamos un paquete (*Package*) con el nombre indicado en la sesión [org.ip.sesion01](#): Botón derecho sobre el nombre del proyecto, y seleccionamos *New > Package*

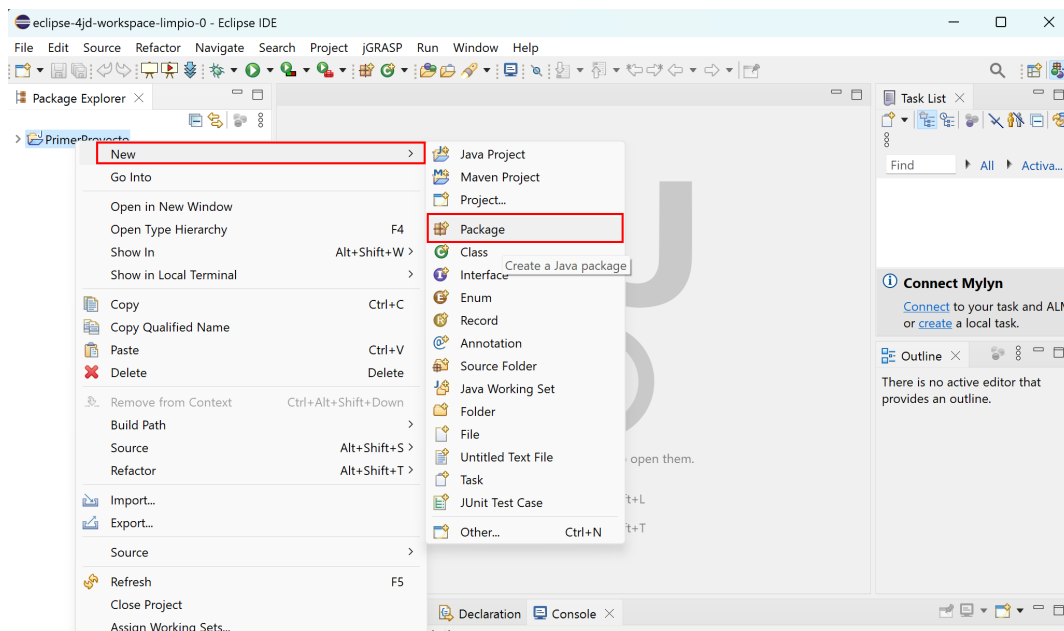


Figura 19. Creación de un paquete

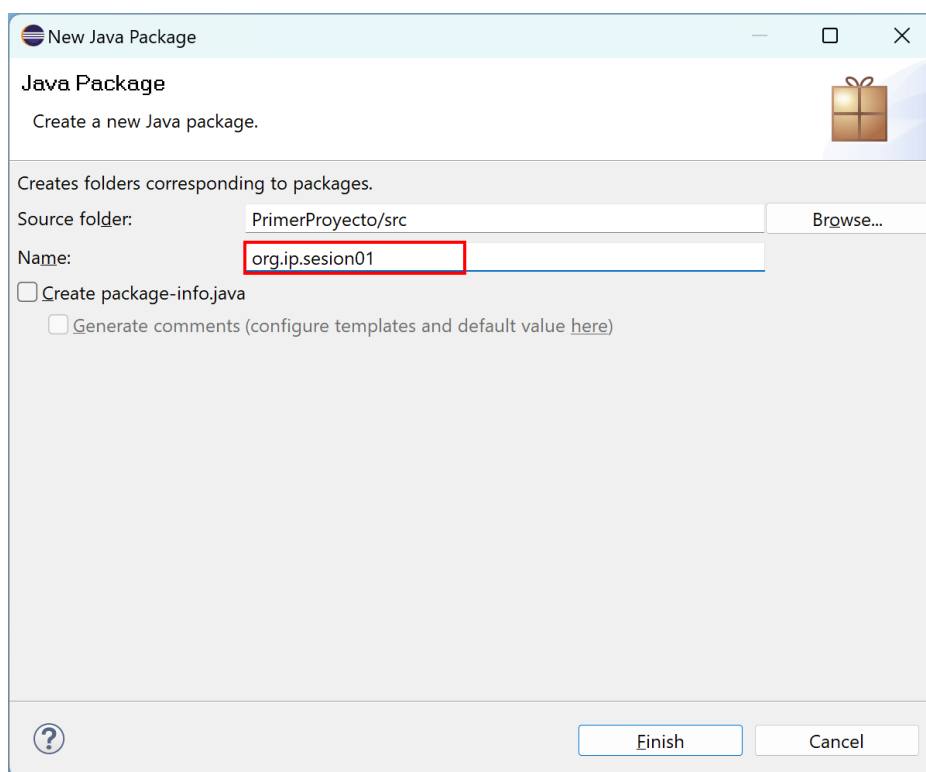


Figura 20. Creación del nombre del paquete



Seguramente este paquete ya existirá en tu proyecto personal que has clonado desde GitHub. Si no existiera, créalo.

2. Dentro del paquete crearemos las clases java. Sobre el nombre del paquete, pulsamos botón derecho y seleccionamos *New > Class*. A continuación, damos el nombre a la clase: *HolaMundo*. Marcamos la opción *public static void main(String args[])* y pulsamos *Finish*.

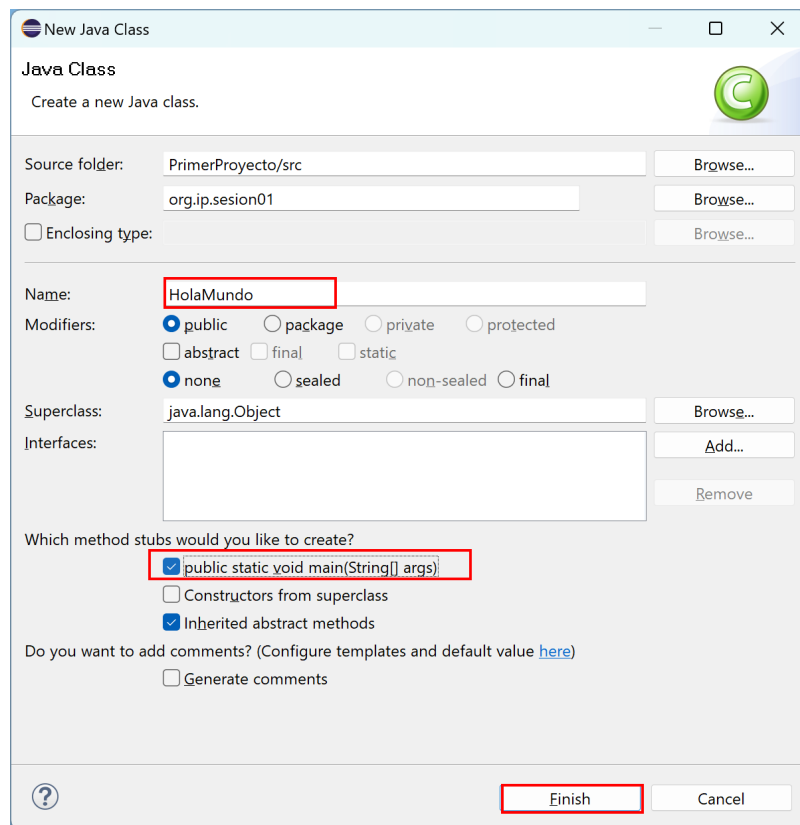


Figura 21. Creación de una clase Java

3. Ya podemos escribir el código de nuestro programa usando el editor de Eclipse

6.5. CTRL + Espacio = Autocompletar

Una vez que conozcas la útil función de autocompletar la usarás continuamente. Se escribirán los primeros caracteres de una sentencia, clase, etc. y a continuación CTRL + Espacio y se autocompletará el resto.

Por ejemplo:

- coloca el cursor en la línea 7 que está en blanco (después de la línea de comentarios)
- empieza escribiendo `sys`
- pulsa CTRL + Espacio. Se autocompletará la sentencia `System.out.println();`
- entre los paréntesis escribe `"Hola Mundo"`

6.6. Compilar y detectar errores

La **compilación** es una tarea que **se lanza automáticamente** al guardar los cambios realizados en el código. Por esta razón, no se va a controlar manualmente. No será necesario pasar por el tedioso y lento proceso de compilar - observar los errores - corregir los errores.

En Eclipse **los errores de compilación se muestran en tiempo real** subrayando el fragmento de código adecuado con una **línea roja**. Los errores pueden encontrarse fácilmente porque se muestran además como marcas rojas en el margen derecho del editor de código Java. Las advertencias (*warnings*) se muestran de la misma manera, pero con marcas amarillas.

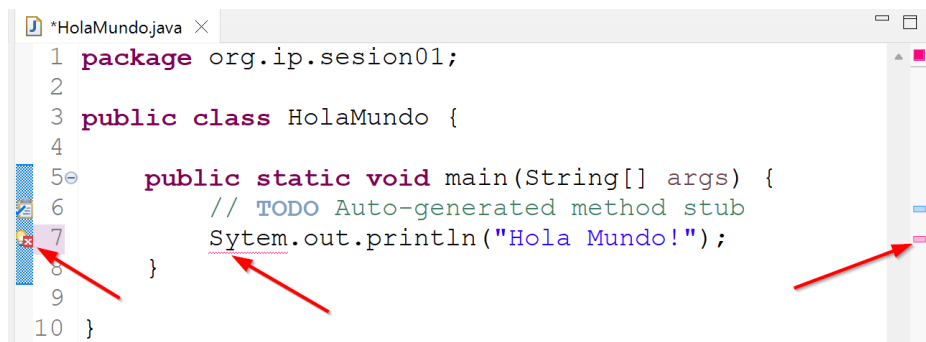


Figura 22. Errores de compilación

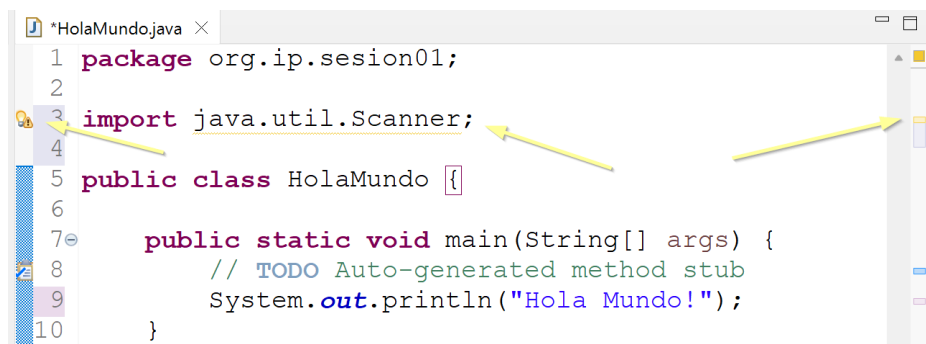


Figura 23. Warnings

6.7. Icono de Bombilla = Autocorregir

Hemos visto como Eclipse detecta y marca todo error y advertencia de compilación. Eclipse habitualmente permite autocorregir los posibles errores haciendo clic en el icono de bombilla presente en el margen izquierdo del editor de código. Así pues, aparecerá una ventana mostrando todas las opciones. Seleccionar una opción mediante los cursores del teclado o dejar el punto del ratón sobre dicha opción abrirá una nueva ventana mostrando detalladamente las modificaciones de código que la autocorrección efectuaría. Basta con pulsar la opción seleccionada (o pulsar ENTER) para hacer que Eclipse lleve a cabo la corrección automatizada.

6.8. Ejecución de una aplicación Java

En el explorador de paquetes, sobre el **nombre de la clase**, pulsamos botón derecho y seleccionamos: *Run As > Java Application*

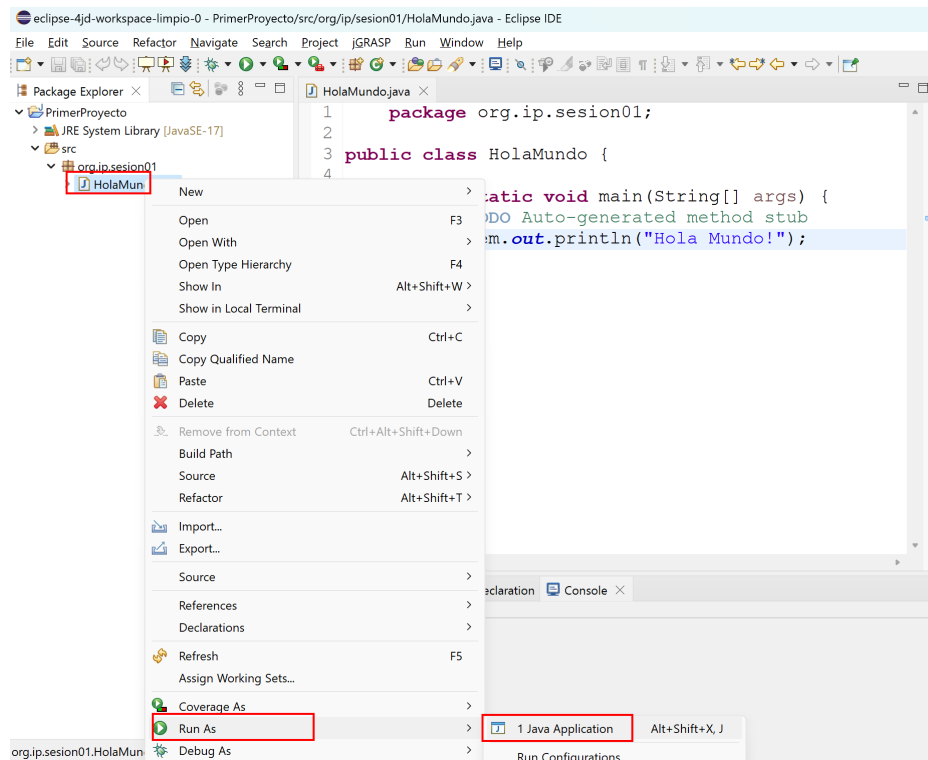


Figura 24. Ejecución de una aplicación Java

La salida que se genera al ejecutar el programa se muestra en la pestaña Console:

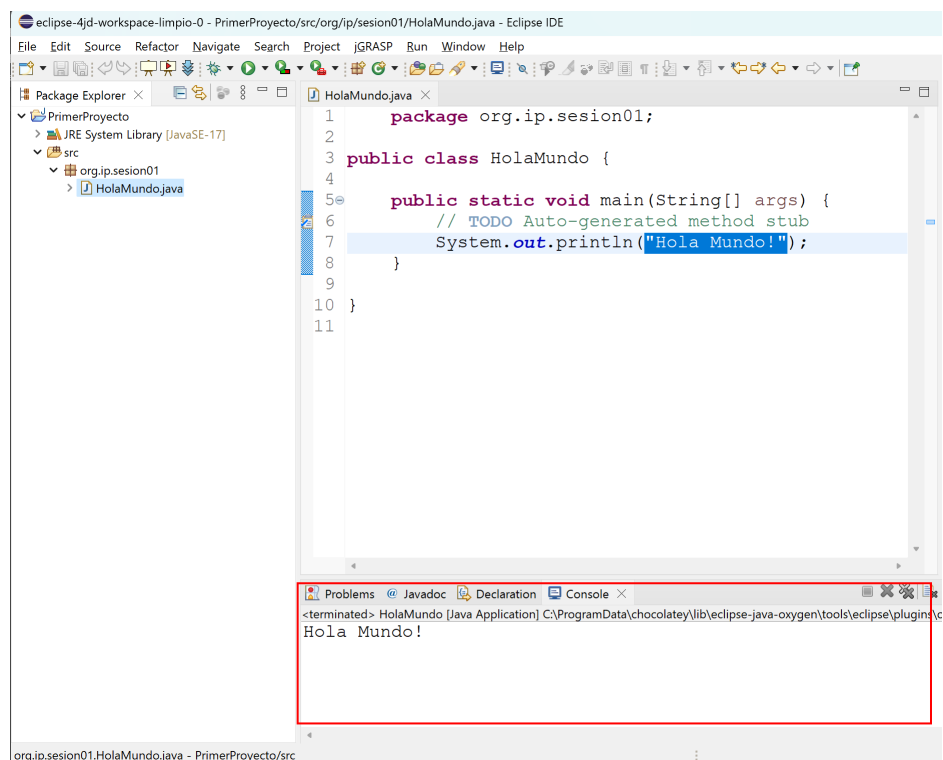


Figura 25. Salida de la ejecución de una aplicación Java

Una vez que has ejecutado un programa Java una primera vez, las siguientes puedes ejecutarlo con el icono *Run* de la barra de iconos.

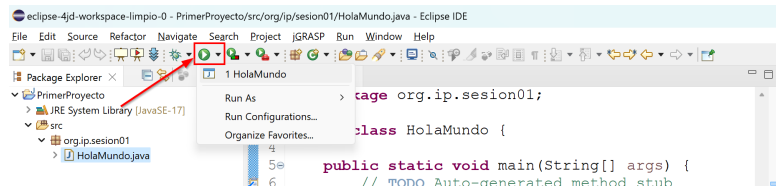


Figura 26. Ejecución de una aplicación Java con icono Run

6.9. Ejecución con argumentos

Los argumentos son valores de entrada que el programa necesita para su ejecución, y que serán almacenados en la variable `args` del método `main()`. Para introducir argumentos: . Modifica el código, la línea donde se muestra "Hola Mundo" modificalá por `System.out.println("Hola " + args[0]);` . A continuación, selecciona el menú de ejecución sobre el nombre de la clase, botón derecho, *Run As > Run Configurations* . En la pestaña *Arguments* , escribe: **MUNDO**

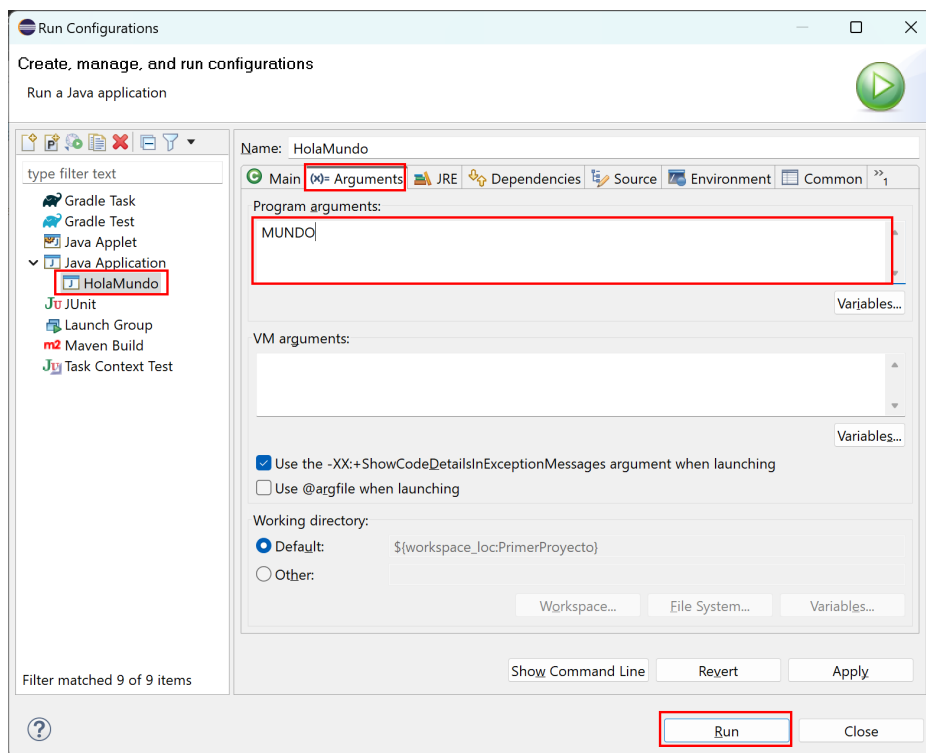


Figura 27. Ejecución de una aplicación Java con argumentos

Salida generada:

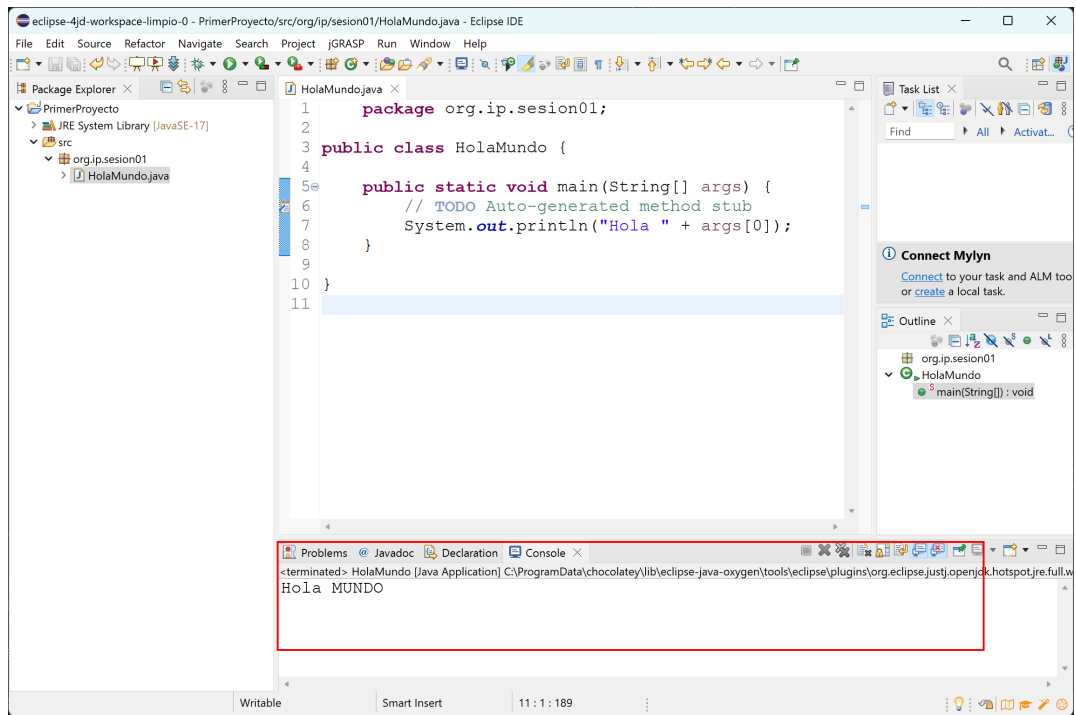


Figura 28. Salida obtenida de la ejecución con argumentos

7. Conclusiones

En este documento has aprendido a instalar Eclipse for Java Developers, a configurar el workspace, a familiarizarte con el entorno de desarrollo Eclipse, a crear un proyecto Java, a crear una clase Java, a compilar y detectar errores, a ejecutar una aplicación Java y a ejecutar una aplicación Java con argumentos.